

Chapter 9: Event, Handling Events Swing

9.1. Event Handling Fundamental

Any operating environment that supports GUIs constantly monitors events such as keystrokes or mouse clicks. The operating environment reports these events to the programs that are running. Each program then decides what, if anything, to do in response to these events. Event handling is fundamental to Java programming because it is integral to the creation of applets and other types of GUI-based programs. Applets are event-driven programs that use a graphical user interface to interact with the user. Furthermore, any program that uses a graphical user interface, such as a Java application written for Windows, is event driven. Thus, we cannot write these types of programs without a solid command of event handling. Events are supported by a number of packages, including *java.util*, *java.awt*, and *java.awt.event*.

Most events to which our program will respond are generated when the user interacts with a **GUI-based program**. They are passed to our program in a variety of ways, with the specific method dependent upon the actual event. There are several types of events, including those generated by the *mouse, the keyboard, and various GUI controls, such as a push button, scroll bar, or check box*.

Events are also occasionally used for purposes not directly related to GUI-based programs. In all cases, the same basic event handling techniques apply.

9.2. The Delegation Event Model

The modern approach to handling events is based on the *delegation event model*, which defines standard and consistent mechanisms to *generate and process events*. Its concept is quite simple: *a source generates an event and sends it to one or more listeners*. In this scheme, *the listener simply waits until it receives an event*. Once an event is received, *the listener processes the event and then returns*. The advantage of this design is that the *application logic that processes events is cleanly separated from the user interface logic that generates those events*. A user interface element is able to “delegate” the processing of an event to a separate piece of code.

In the delegation event model, *listeners must register with a source in order to receive an event notification*. This provides an important benefit: *notifications are sent only to listeners that want to receive them*. This is a more efficient way to handle events than the design used by the old Java 1.0 approach. Previously, an event was propagated up the containment hierarchy until it was handled by a component. This required components to receive events that they did not process, and it wasted valuable time. The delegation event model eliminates this overhead.

Java also allows us to process events without using the delegation event model. *Extending an AWT component can do this*. However, the delegation event model is the preferred design for the reasons just cited.

9.3. Events

In the *delegation* model, an event is an *object* that describes a state change in a source. It can be generated as a consequence of a person interacting with the elements in a graphical user interface. Some of the activities that cause events to be generated are *pressing a button, entering a character via the keyboard, selecting an item in a list, and clicking the mouse*.

Events may also occur that are not directly caused by interactions with a user interface. For example, an event may be generated when a *timer expires, a counter exceeds a value, software or hardware failure occurs, or an operation is completed*. We are free to define events that are appropriate for our application.

9.3.1. Event Sources

A source is an *object* that generates an event. This occurs when the internal state of that object changes in some way. Sources may generate more than one type of event. *A source must register listeners in order for the listeners to receive notifications about a specific type of event.* Each type of event has its own registration method. Here is the general form:

```
public void addTypeListener(TypeListener el)
```

Here, Type is the name of the event, and el is a reference to the event listener. For example, the method that registers a keyboard event listener is called *addKeyListener()*. The method that registers a mouse motion listener is called *addMouseMotionListener()*. When an event occurs, all registered listeners are notified and receive a copy of the event object. *This is known as multicasting the event.* In all cases, notifications are sent only to listeners that register to receive them.

Some sources may allow only one listener to register. The general form of such a method is this:

```
public void addTypeListener(TypeListener el)
```

```
throws java.util.TooManyListenersException
```

Here, Type is the name of the event, and el is a reference to the event listener. When such an event occurs, the registered listener is notified. *This is known as unicasting the event.* A source must also provide a method that allows a listener to unregister an interest in a specific type of event. The general form of such a method is this:

```
public void removeTypeListener(TypeListener el)
```

Here, Type is the name of the event, and el is a reference to the event listener. For example, to remove a keyboard listener, we would call *removeKeyListener()*. The methods that add or remove listeners are provided by the source that generates events. For example, the **Component class** provides methods to add and remove keyboard and mouse event listeners.

9.3.2. Event Listeners

A listener is an object that is notified when an event occurs. It has two major requirements. First, it must have been *registered with one or more sources to receive notifications about specific types of events.* Second, it *must implement methods to receive and process these notifications.*

The methods that receive and process events are defined in a set of interfaces found in *java.awt.event*. For example, the *MouseMotionListener* interface defines two methods to receive notifications when the mouse is dragged or moved. Any object may receive and process one or both of these events if it provides an implementation of this interface.

9.4. Event Classes

The classes that represent events are at the core of Java's *event handling mechanism*. Thus, a discussion of event handling must begin with the event classes. The most widely used events are those defined by the AWT and those defined by Swing.

At the root of the Java event class hierarchy is *EventObject*, which is in *java.util*. It is the superclass for all events. Its one constructor is shown here:

```
EventObject(Object src)
```

Here, *src* is the object that generates this event. *EventObject* contains two methods: *getSource()* and *toString()*. The *getSource()* method returns the source of the event. Its general form is shown here: *Object getSource()*. As expected, *toString()* returns the string equivalent of the event.

The class *AWTEvent*, defined within the *java.awt* package, is a subclass of *EventObject*.

It is the superclass (either directly or indirectly) of all AWT-based events used by the delegation event model. Its *getID()* method can be used to determine the type of the event. The signature of this method is shown here:

```
int getID()
```

To summarize:

- `EventObject` is a superclass of all events.
- ***AWTEvent*** is a superclass of all AWT events that are handled by the delegation event model.

The package `java.awt.event` defines many types of events that are generated by various user interface elements. Following table shows several commonly used event classes and provides a brief description of when they are generated.

Event Class	Description
<code>ActionEvent</code>	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
<code>AdjustmentEvent</code>	Generated when a scroll bar is manipulated.
<code>ComponentEvent</code>	Generated when a component is hidden, moved, resized, or becomes visible.
<code>ContainerEvent</code>	Generated when a component is added to or removed from a container.
<code>FocusEvent</code>	Generated when a component gains or loses keyboard focus.
<code>InputEvent</code>	Abstract superclass for all component input event classes.
<code>ItemEvent</code>	Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
<code>KeyEvent</code>	Generated when input is received from the keyboard.
<code>MouseEvent</code>	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
<code>MouseWheelEvent</code>	Generated when the mouse wheel is moved.
<code>TextEvent</code>	Generated when the value of a text area or text field is changed.
<code>WindowEvent</code>	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Fig: Main Event Classes in `java.awt.event`

9.4.1. The `ActionEvent` Class

An `ActionEvent` is generated when a *button is pressed, a list item is double-clicked, or a menu item is selected*. The `ActionEvent` class defines four integer constants that can be used to identify any modifiers associated with an action event: `ALT_MASK`, `CTRL_MASK`, `META_MASK`, and `SHIFT_MASK`. In addition, there is an integer constant, `ACTION_PERFORMED`, which can be used to identify action events.

`ActionEvent` has these three constructors:

`ActionEvent(Object src, int type, String cmd)`

`ActionEvent(Object src, int type, String cmd, int modifiers)`

`ActionEvent(Object src, int type, String cmd, long when, int modifiers)`

Here, `src` is a reference to the object that generated this event. The type of the event is specified by `type`, and its command string is `cmd`. The argument `modifiers` indicates which modifier keys (`ALT`, `CTRL`, `META`, and/or `SHIFT`) were pressed when the event was generated. The `when` parameter specifies when the event occurred. We can obtain the command name for the invoking `ActionEvent` object by using the `getActionCommand()` method, shown here:

`String getActionCommand()`

For example, when a button is pressed, an action event is generated that has a command name equal to the label on that button.

The `getModifiers()` method returns a value that indicates which modifier keys (`ALT`, `CTRL`, `META`, and/or `SHIFT`) were pressed when the event was generated. Its form is shown here:

int getModifiers()

The method *getWhen()* returns the time at which the event took place. This is called the event's timestamp. The *getWhen()* method is shown here:

long getWhen()

9.4.2. The AdjustmentEvent Class

An *AdjustmentEvent* is generated by a scroll bar. There are five types of adjustment events. The *AdjustmentEvent* class defines integer constants that can be used to identify them. The constants and their meanings are shown here:

BLOCK_DECREMENT	The user clicked inside the scroll bar to decrease its value.
BLOCK_INCREMENT	The user clicked inside the scroll bar to increase its value.
TRACK	The slider was dragged.
UNIT_DECREMENT	The button at the end of the scroll bar was clicked to decrease its value.
UNIT_INCREMENT	The button at the end of the scroll bar was clicked to increase its value.

In addition, there is an integer constant, *ADJUSTMENT_VALUE_CHANGED*, that indicates that a change has occurred. Here is one *AdjustmentEvent* constructor:

AdjustmentEvent(Adjustable src, int id, int type, int data)

Here, *src* is a reference to the object that generated this event. The *id* specifies the event. The *type* of the adjustment is specified by *type*, and its associated data is *data*.

The *getAdjustable()* method returns the object that generated the event. Its form is shown here:

Adjustable getAdjustable()

The type of the adjustment event may be obtained by the *getAdjustmentType()* method. It returns one of the constants defined by *AdjustmentEvent*. The general form is shown here:

int getAdjustmentType()

The amount of the adjustment can be obtained from the *getValue()* method, shown here:

int getValue()

For example, when a scroll bar is manipulated, this method returns the value represented by that change.

9.4.3. The ComponentEvent Class

A *ComponentEvent* is generated when the *size*, *position*, or *visibility* of a component is changed. There are four types of component events. The *ComponentEvent* class defines integer constants that can be used to identify them.

COMPONENT_HIDDEN	The component was hidden.
COMPONENT_MOVED	The component was moved.
COMPONENT_RESIZED	The component was resized.
COMPONENT_SHOWN	The component became visible.

ComponentEvent has this constructor:

ComponentEvent(Component src, int type)

Here, *src* is a reference to the object that generated this event. The type of the event is specified by *type*.

ComponentEvent is the superclass either directly or indirectly of *ContainerEvent*, *FocusEvent*, *KeyEvent*, *MouseEvent*, and *WindowEvent*.

The *getComponent()* method returns the component that generated the event.

Component getComponent()

9.4.4. The ContainerEvent Class

A *ContainerEvent* is generated when a component is added to or removed from a container.

There are two types of container events. The *ContainerEvent* class defines int constants that can be used to identify them: *COMPONENT_ADDED* and *COMPONENT_REMOVED*. They indicate that a component has been added to or removed from the container.

ContainerEvent is a subclass of *ComponentEvent* and has this constructor:

ContainerEvent(Component src, int type, Component comp)

Here, *src* is a reference to the container that generated this event. The type of the event is specified by *type*, and the component that has been added to or removed from the container is *comp*.

We can obtain a reference to the container that generated this event by using the

getContainer() method, shown here:

Container getContainer()

The *getChild()* method returns a reference to the component that was added to or removed from the container. Its general form is shown here:

Component getChild()

9.4.5. The FocusEvent Class

A *FocusEvent* is generated when a component gains or loses input focus. These events are identified by the integer constants *FOCUS_GAINED* and *FOCUS_LOST*.

FocusEvent is a subclass of *ComponentEvent* and has these constructors:

FocusEvent(Component src, int type)

FocusEvent(Component src, int type, boolean temporaryFlag)

FocusEvent(Component src, int type, boolean temporaryFlag, Component other)

Here, *src* is a reference to the component that generated this event. The type of the event is specified by *type*. The argument *temporaryFlag* is set to **true** if the focus event is *temporary*. Otherwise, it is set to false. (A temporary focus event occurs as a result of another user interface operation. For example, assume that the focus is in a text field. If the user moves the mouse to adjust a scroll bar, the focus is temporarily lost.)

The other component involved in the focus change, called the opposite component, is passed in *other*. Therefore, if a *FOCUS_GAINED* event occurred, *other* will refer to the component that lost focus. Conversely, if a *FOCUS_LOST* event occurred, *other* will refer to the component that gains focus.

We can determine the other component by calling *getOppositeComponent()*,

Component getOppositeComponent(). The opposite component is returned.

The *isTemporary()* method indicates if this focus change is temporary.

boolean isTemporary(). The method returns true if the change is temporary. Otherwise, it returns false.

9.4.6. The InputEvent Class

The abstract class *InputEvent* is a subclass of *ComponentEvent* and is the superclass for component input events. Its subclasses are *KeyEvent* and *MouseEvent*.

InputEvent defines several integer constants that represent any modifiers, such as the control key being pressed, that might be associated with the event. Originally, the *InputEvent* class defined the following eight values to represent the modifiers:

ALT_MASK	BUTTON2_MASK	META_MASK
ALT_GRAPH_MASK	BUTTON3_MASK	SHIFT_MASK
BUTTON1_MASK	CTRL_MASK	

However, because of possible conflicts between the modifiers used by keyboard events and mouse events, and other issues, the following extended modifier values were added:

ALT_DOWN_MASK	BUTTON2_DOWN_MASK	META_DOWN_MASK
ALT_GRAPH_DOWN_MASK	BUTTON3_DOWN_MASK	SHIFT_DOWN_MASK
BUTTON1_DOWN_MASK	CTRL_DOWN_MASK	

When writing new code, it is recommended that we use the new, extended modifiers rather than the original modifiers.

To test if a modifier was pressed at the time an event is generated, use the *isAltDown()*, *isAltGraphDown()*, *isControlDown()*, *isMetaDown()*, and *isShiftDown()* methods. The forms of these methods are shown here:

boolean isAltDown()

boolean isAltGraphDown()

boolean isControlDown()

boolean isMetaDown()

boolean isShiftDown()

We can obtain a value that contains all of the original modifier flags by calling the *getModifiers()* method.

int getModifiers()

We can obtain the extended modifiers by calling *getModifiersEx()*, which is shown here:

int getModifiersEx()

9.4.7. The ItemEvent Class

An *ItemEvent* is generated when a *check box* or a *list item* is clicked or when a *checkable menu item* is selected or deselected. There are two types of item events, which are identified by the following integer constants:

DESELECTED	The user deselected an item.
SELECTED	The user selected an item.

In addition, *ItemEvent* defines one integer constant, *ITEM_STATE_CHANGED*, that signifies a change of state. *ItemEvent* has this constructor:

ItemEvent(ItemSelectable src, int type, Object entry, int state)

Here, *src* is a reference to the component that generated this event. For example, this might be a *list* or *choice element*. The type of the event is specified by *type*. The specific item that generated the item event is passed in *entry*. The current state of that item is in *state*.

The *getItem()* method can be used to obtain a reference to the item that generated an event.

Object getItem()

The *getItemSelectable()* method can be used to obtain a reference to the *ItemSelectable* object that generated an event.

ItemSelectable getItemSelectable()

Lists and choices are examples of user interface elements that implement the *ItemSelectable* interface.

The *getStateChange()* method returns the state change (that is, *SELECTED* or *DESELECTED*) for the event.

int getStateChange()

9.4.8. The KeyEvent Class

A *KeyEvent* is generated when keyboard input occurs. There are three types of key events, which are identified by these integer constants: *KEY_PRESSED*, *KEY_RELEASED*, and *KEY_TYPED*. The first two events are generated when any key is pressed or released. The last event occurs only when a character is generated.

There are many other integer constants that are defined by *KeyEvent*. For example, *VK_0* through *VK_9*

and VK_A through VK_Z define the ASCII equivalents of the numbers and letters. Here are some others:

VK_ALT	VK_DOWN	VK_LEFT	VK_RIGHT
VK_CANCEL	VK_ENTER	VK_PAGE_DOWN	VK_SHIFT
VK_CONTROL	VK_ESCAPE	VK_PAGE_UP	VK_UP

The VK constants specify virtual key codes and are independent of any modifiers, such as *control*, *shift*, or *alt*. *KeyEvent* is a subclass of *InputEvent*. Here is one of its constructors:

KeyEvent(Component src, int type, long when, int modifiers, int code, char ch)

Here, *src* is a reference to the component that generated the event. The type of the event is specified by *type*. The system time at which the key was pressed is passed in *when*. The *modifiers* argument indicates which modifiers were pressed when this key event occurred. The virtual key code, such as VK_UP, VK_A, and so forth, is passed in *code*. The character equivalent (if one exists) is passed in *ch*. If no valid character exists, then *ch* contains CHAR_UNDEFINED.

For KEY_TYPED events, code will contain VK_UNDEFINED.

The *KeyEvent* class defines several methods, but the most commonly used ones are *getKeyChar()*, which returns the character that was entered, and *getKeyCode()*, which returns the key code.

char getKeyChar()

int getKeyCode()

If no valid character is available, then *getKeyChar()* returns CHAR_UNDEFINED. When a KEY_TYPED event occurs, *getKeyCode()* returns VK_UNDEFINED.

9.4.9. The MouseEvent Class

There are eight types of mouse events. The *MouseEvent* class defines the following integer constants that can be used to identify them:

MOUSE_CLICKED	The user clicked the mouse.
MOUSE_DRAGGED	The user dragged the mouse.
MOUSE_ENTERED	The mouse entered a component.
MOUSE_EXITED	The mouse exited from a component.
MOUSE_MOVED	The mouse moved.
MOUSE_PRESSED	The mouse was pressed.
MOUSE_RELEASED	The mouse was released.
MOUSE_WHEEL	The mouse wheel was moved.

MouseEvent is a subclass of *InputEvent*. Here is one of its constructors:

MouseEvent(Component src, int type, long when, int modifiers, int x, int y, int clicks, boolean triggersPopup)

Here, *src* is a reference to the component that generated the event. The type of the event is specified by *type*. The system time at which the mouse event occurred is passed in *when*. The *modifiers* argument indicates which modifiers were pressed when a mouse event occurred. The coordinates of the mouse are passed in *x* and *y*. The click count is passed in *clicks*. The *triggersPopup* flag indicates if this event causes a pop-up menu to appear on this platform.

Two commonly used methods in this class are *getX()* and *getY()*. These return the X and Y coordinates of the mouse within the component when the event occurred.

int getX()

int getY()

Alternatively, we can use the *getPoint()* method to obtain the coordinates of the mouse.

Point getPoint(). It returns a *Point* object that contains the X,Y coordinates in its integer members: *x* and *y*.

The *translatePoint()* method changes the location of the event.

void translatePoint(int x, int y). Here, the arguments x and y are added to the coordinates of the event.

The *getClickCount()* method obtains the number of mouse clicks for this event.

int getClickCount()

The *isPopupTrigger()* method tests if this event causes a pop-up menu to appear on this platform.

boolean isPopupTrigger()

Also available is the *getButton()* method.

int getButton()

It returns a value that represents the button that caused the event. The return value will be one of these constants defined by *MouseEvent*:

NOBUTTON	BUTTON1	BUTTON2	BUTTON3
----------	---------	---------	---------

The NOBUTTON value indicates that no button was pressed or released. Java SE 6 added three methods to *MouseEvent* that obtain the coordinates of the mouse relative to the screen rather than the component.

Point getLocationOnScreen()

int getXOnScreen()

int getYOnScreen()

The *getLocationOnScreen()* method returns a *Point* object that contains both the X and Y coordinate. The other two methods return the indicated coordinate.

9.4.10. The *MouseWheelEvent* Class

The *MouseWheelEvent* class encapsulates a mouse wheel event. It is a subclass of *MouseEvent*. Not all mice have wheels. If a mouse has a wheel, it is located between the left and right buttons. Mouse wheels are used for scrolling. *MouseWheelEvent* defines these two integer constants:

WHEEL_BLOCK_SCROLL	A page-up or page-down scroll event occurred.
WHEEL_UNIT_SCROLL	A line-up or line-down scroll event occurred.

Here is one of the constructors defined by *MouseWheelEvent*:

MouseWheelEvent(Component src, int type, long when, int modifiers, int x, int y, int clicks, boolean triggersPopup, int scrollHow, int amount, int count)

Here, *src* is a reference to the object that generated the event. The type of the event is specified by *type*. The system time at which the mouse event occurred is passed in *when*. The *modifiers* argument indicates which modifiers were pressed when the event occurred. The coordinates of the mouse are passed in x and y. The number of clicks the wheel has rotated is passed in *clicks*. The *triggersPopup* flag indicates if this event causes a pop-up menu to appear on this platform. The *scrollHow* value must be either WHEEL_UNIT_SCROLL or WHEEL_BLOCK_SCROLL. The number of units to scroll is passed in *amount*. The count parameter indicates the number of rotational units that the wheel moved.

9.4.11. The *TextEvent* Class

Instances of this class describe text events. These are generated by *text fields* and *text areas* when characters are entered by a user or program. *TextEvent* defines the integer constant TEXT_VALUE_CHANGED.

The one constructor for this class is shown here:

TextEvent(Object src, int type)

Here, *src* is a reference to the object that generated this event. The type of the event is specified by *type*.

The *TextEvent* object does not include the characters currently in the text component that generated the event. Instead, our program must use other methods associated with the text component to retrieve that information.

9.4.12. The WindowEvent Class

There are ten types of window events. *TheWindowEvent* class defines integer constants that can be used to identify them. The constants and their meanings are shown in below table:

WindowEvent is a subclass of *ComponentEvent*. It defines several constructors. The first is

WindowEvent(Window src, int type)

WindowEvent(Window src, int type, Window other)

WindowEvent(Window src, int type, int fromState, int toState)

WindowEvent(Window src, int type, Window other, int fromState, int toState)

WINDOW_ACTIVATED	The window was activated.
WINDOW_CLOSED	The window has been closed.
WINDOW_CLOSING	The user requested that the window be closed.
WINDOW_DEACTIVATED	The window was deactivated.
WINDOW_DEICONIFIED	The window was deiconified.
WINDOW_GAINED_FOCUS	The window gained input focus.
WINDOW_ICONIFIED	The window was iconified.
WINDOW_LOST_FOCUS	The window lost input focus.
WINDOW_OPENED	The window was opened.
WINDOW_STATE_CHANGED	The state of the window changed.

Here, *other* specifies the opposite window when a focus or activation event occurs. The *fromState* specifies the prior state of the window, and *toState* specifies the new state that the window will have when a window state change occurs.

A commonly used method in this class is *getWindow()*. It returns the *Window* object that generated the event.

Window getWindow()

WindowEvent also defines methods that return the opposite window (when a focus or activation event has occurred), the previous window state, and the current window state.

Window getOppositeWindow()

int getOldState()

int getNewState()

9.5. Sources of Events

Following table lists some of the user interface components that can generate the events. In addition to these graphical user interface elements, any class derived from *Component*, such as *Applet*, can generate events. For example, we can receive key and mouse events from an applet.

Event Source	Description
Button	Generates action events when the button is pressed.
Check box	Generates item events when the check box is selected or deselected.
Choice	Generates item events when the choice is changed.
List	Generates action events when an item is double-clicked; generates item events when an item is selected or deselected.
Menu Item	Generates action events when a menu item is selected; generates item events when a checkable menu item is selected or deselected.
Scroll bar	Generates adjustment events when the scroll bar is manipulated.
Text components	Generates text events when the user enters a character.
Window	Generates window events when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Table: Sources of Events

9.6. Event Listener Interfaces

The delegation event model has two parts: *sources* and *listeners*. Listeners are created by implementing one or more of the interfaces defined by the *java.awt.event* package. When an event occurs, the event source invokes the appropriate method defined by the *listener* and provides *an event object* as its argument. Below table lists commonly used *listener* interfaces and provides a brief description of the methods that they define.

Interface	Description
ActionListener	Defines one method to receive action events.
AdjustmentListener	Defines one method to receive adjustment events.
ComponentListener	Defines four methods to recognize when a component is hidden, moved, resized, or shown.
ContainerListener	Defines two methods to recognize when a component is added to or removed from a container.
FocusListener	Defines two methods to recognize when a component gains or loses keyboard focus.
ItemListener	Defines one method to recognize when the state of an item changes.
KeyListener	Defines three methods to recognize when a key is pressed, released, or typed.
MouseListener	Defines five methods to recognize when the mouse is clicked, enters a component, exits a component, is pressed, or is released.
MouseMotionListener	Defines two methods to recognize when the mouse is dragged or moved.
MouseWheelListener	Defines one method to recognize when the mouse wheel is moved.
TextListener	Defines one method to recognize when a text value changes.
WindowFocusListener	Defines two methods to recognize when a window gains or loses input focus.
WindowListener	Defines seven methods to recognize when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Table: Commonly Used Event Listener Interfaces

9.7. Handling Mouse Event

Handling mouse event using the delegation event model is actually quite easy. Just follow these two steps:

1. Implement the appropriate interface in the listener so that it will receive the type of event desired.
2. Implement code to register and unregister (if necessary) the listener as a recipient for the event notifications.

Remember that a source may generate several types of events. Each event must be registered separately. Also, an object may register to receive several types of events, but it must implement all of the interfaces that are required to receive these events.

To handle mouse events, we must implement the *MouseListener* and the *MouseMotionListener* interfaces. The following applet demonstrates the process. It displays the current coordinates of the mouse in the applet's status window. Each time a button is pressed, the word "Down" is displayed at the location of the mouse pointer. Each time the button is released, the word "Up" is shown. If a button is clicked, the message "Mouse clicked" is displayed in the upperleft corner of the applet display area.

As the mouse enters or exits the applet window, a message is displayed in the upper-left corner of the applet display area. When dragging the mouse, a * is shown, which tracks with the mouse pointer as it is dragged. Notice that the two variables, *mouseX* and *mouseY*, store the location of the mouse when a *mouse pressed, released, or dragged event occurs*. These coordinates are then used by *paint()* to display output at the point of these occurrences.

// Demonstrate the mouse event handlers.

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="MouseEvents" width=300 height=100>
</applet>
*/
public class MouseEvents extends Applet implements MouseListener,
MouseMotionListener {
    String msg = "";
    int mouseX = 0, mouseY = 0; // coordinates of mouse
    public void init() {
        addMouseListener(this);
        addMouseMotionListener(this);
    }
    // Handle mouse clicked.
    public void mouseClicked(MouseEvent me) {
        // save coordinates
        mouseX = 0;
        mouseY = 10;
        msg = "Mouse clicked.";
        repaint();
    }
    // Handle mouse entered.
    public void mouseEntered(MouseEvent me) {
        // save coordinates
        mouseX = 0;
        mouseY = 10;
        msg = "Mouse entered.";
        repaint();
    }
    // Handle mouse exited.
    public void mouseExited(MouseEvent me) {
        // save coordinates
        mouseX = 0;
        mouseY = 10;
        msg = "Mouse exited.";
        repaint();
    }
    // Handle button pressed.
    public void mousePressed(MouseEvent me) {
        // save coordinates
        mouseX = me.getX();
        mouseY = me.getY();
        msg = "Down";
        repaint();
    }
    // Handle button released.
    public void mouseReleased(MouseEvent me) {
        // save coordinates
        mouseX = me.getX();
        mouseY = me.getY();
        msg = "Up";
        repaint();
    }
    // Handle mouse dragged.
    public void mouseDragged(MouseEvent me) {
```

```
// save coordinates
mouseX = me.getX();
mouseY = me.getY();
msg = "*";
showStatus("Dragging mouse at " + mouseX + ", " + mouseY);
repaint();
}
// Handle mouse moved.
public void mouseMoved(MouseEvent me) {
    // show status
    showStatus("Moving mouse at " + me.getX() + ", " + me.getY());
}
// Display msg in applet window at current X,Y location.
public void paint(Graphics g) {
    g.drawString(msg, mouseX, mouseY);
}
}
```

The **MouseEvents** class extends **Applet** and implements both the **MouseListener** and **MouseMotionListener** interfaces. These two interfaces contain methods that receive and process the various types of mouse events. Notice that the **applet** is both the source and the listener for these events. This works because **Component**, which supplies the **addMouseListener()** and **addMouseMotionListener()** methods, is a superclass of **Applet**. Being both the source and the listener for events is a common situation for applets. Inside **init()**, the applet registers itself as a listener for mouse events. This is done by using **addMouseListener()** and **addMouseMotionListener()**, which, are the members of **Component**.

9.8. Swing

Swing is a set of classes that provides more powerful and flexible GUI components than does the AWT. Simply put, Swing provides the *look and feel of the modern Java GUI*. Although Swing eliminates a number of the limitations inherent in the AWT, Swing does not replace it. Instead, *Swing is built on the foundation of the AWT*. This is why the AWT is still a crucial part of Java. Swing also uses the same event handling mechanism as the AWT. Therefore, a basic understanding of the AWT and of event handling is required to use Swing.

Swing was created to address the limitations present in the AWT. It does this through two key features: **lightweight components and a pluggable look and feel**. Together they provide an elegant, yet easy-to-use solution to the problems of the AWT. More than anything else, it is these two features that define the essence of Swing.

9.8.1. Swing Components Are Lightweight

With very few exceptions, Swing components are *lightweight*. This means that they are written entirely in Java and do *not map directly to platform-specific peers*. Because lightweight components are rendered using graphics primitives, they can be *transparent*, which enables nonrectangular shapes. Thus, lightweight components are more efficient and more flexible. Furthermore, because lightweight components do not translate into native peers, the *look and feel of each component is determined by Swing, not by the underlying operating system*. This means that each component will work in a consistent manner across all platforms.

9.8.2. Swing Supports a Pluggable Look and Feel

Swing supports a pluggable look and feel (PLAF). Because each Swing component is *rendered by Java code* rather than by native peers, the look and feel of a component is under the control of Swing. This fact means that it is possible to separate the *look and feel* of a component from the *logic of the component*, and this is what Swing does. Separating out the look and feel provides a significant advantage: *it becomes*

possible to change the way that a component is rendered without affecting any of its other aspects. In other words, it is possible to “plug in” a new look and feel for any given component without creating any side effects in the code that uses that component. Moreover, it becomes possible to define entire sets of look-and-feels that represent different GUI styles. To use a specific style, its look and feel is simply “plugged in.” Once this is done, all components are automatically rendered using that style.

Pluggable look-and-feels offer several important advantages. *It is possible to define a look and feel that is consistent across all platforms.* Conversely, it is possible to create a look and feel that acts like a specific platform. For example, if we know that an application will be running only in a Windows environment, it is possible to specify the Windows look and feel. It is also possible to design a custom look and feel. Finally, the look and feel can be changed dynamically at run time.

9.9. Separating GUI and Application Code (MVC Pattern)

User interface component such as a button, a check box, a text field, or a sophisticated tree control. Every component has three characteristics:

- Its *contents*, such as the state of a button (pushed in or not), or the text in a text field;
- Its *visual appearance* (color, size, and so on);
- Its *behavior* (reaction to events).

Even a seemingly simple component such as a button exhibits some moderately complex interaction between these characteristics. Obviously, the *visual appearance* of a button depends on the look and feel. The appearance depends on the *button state*: *when a button is pushed in, it needs to be redrawn to look different. The state depends on the events that the button receives.* When the user depresses the mouse inside the button, the button is pushed in. When we use a button in our programs, we simply consider it as a button, and we don't think too much about the inner workings and characteristics.

The Swing designers turned to a well-known design pattern: the ***model-view controller*** pattern. This pattern, like many other design patterns, goes back to one of the principles of object-oriented design. Don't make one object responsible for too much. Don't have a single button class do everything. Instead, have the look and feel of the component associated with one object and store the contents in another object. The model-view-controller (MVC) design pattern teaches how to accomplish this. Implement three separate classes:

- *The model, which stores the contents;*
- *The view, which displays the contents;*
- *The controller, which handles user input.*

The pattern specifies precisely how these three objects interact. The *model* stores the contents and has no user interface. For a button, the content is pretty trivial-it is just a small set of flags that tells whether the button is currently pushed in or out, whether it is active or inactive, and so on. The model must implement methods to change the contents and to discover what the contents are.

One of the advantages of the model-view-controller pattern is that a *model* can have multiple *views*, each showing a different part or aspect of the full contents. When the model is updated through the *controller* of one of the views, it tells both attached *views* about the change. When the *views* are notified, they refresh themselves automatically. Of course, for a simple user interface component such as a button, we won't have multiple views of the same model.

The *controller* handles the user input events such as mouse clicks and keystrokes. It then decides whether to translate these events into changes in the *model* or the *view*. For example, if the user presses a character key in a text box, the controller calls the “insert character” command of the model. *The model then tells the view to update itself. The view never knows why the text changed.* But if the user presses a key, then

the controller may tell the view to scroll. Scrolling the view has no effect on the underlying text, so the model never knows that this event happened.

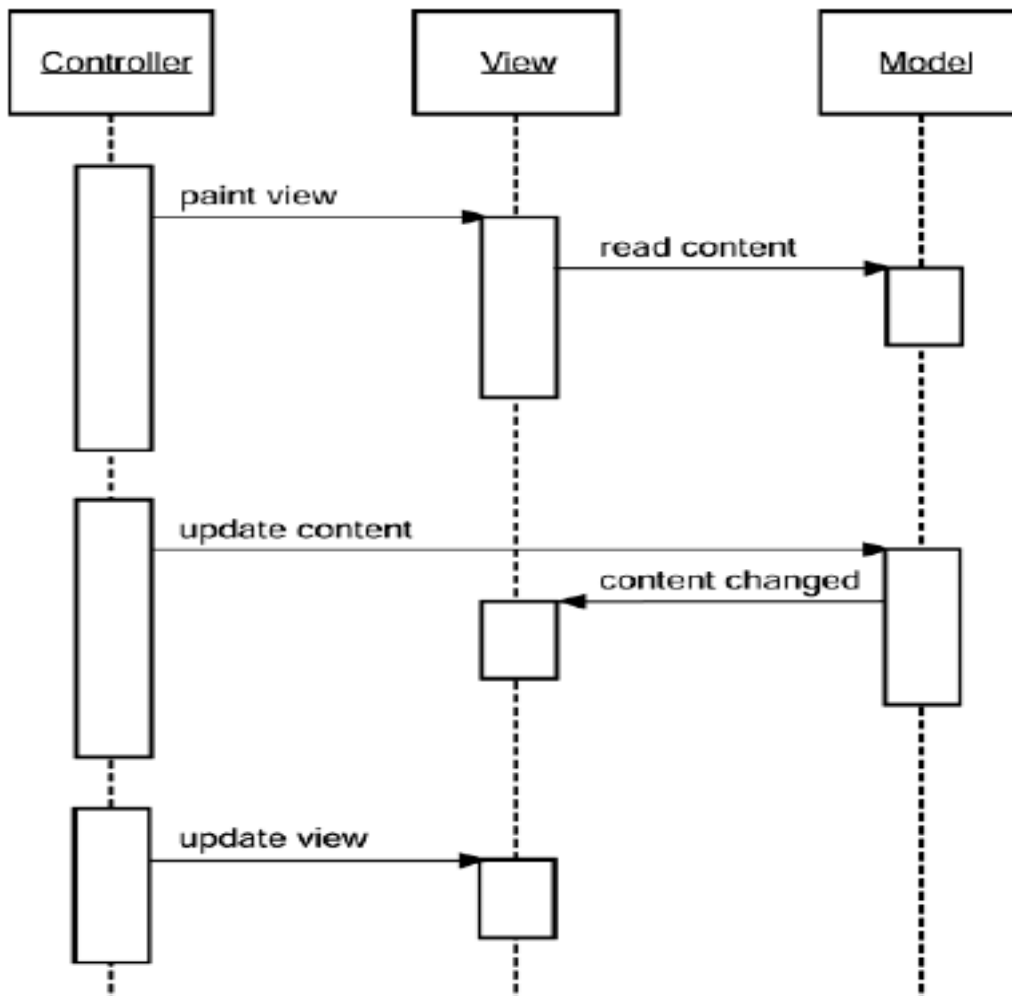


Fig: Interaction Between Model, View and Controller objects

As a programmer using Swing components, we generally don't need to think about the model-view-controller architecture. Each user interface has a *wrapper class* (such as `JButton` or `TextField`) that stores the model and the view. When we want to inquire about the contents (for example, the text in a text field), the *wrapper class asks the model and returns the answer to us*. When we want to change the view (for ex-ample, move the caret position in a text field), the *wrapper class forwards that request to the view*. However, there are occasions where the wrapper class doesn't work hard enough on forwarding commands. Then, we have to ask it to retrieve the model and work directly with the model. (We don't have to work directly with the view-that is the job of the look-and-feel code.)

Besides being “*the right thing to do*,” the **model-view-controller** pattern was attractive for the Swing designers because it allowed them to implement pluggable look and feel. The model of a button or text field is independent of the look-and-feel. But of course the visual representation is completely dependent on the user interface design of a particular look and feel. The controller can vary as well.

9.9.1. A Model-View-Controller Analysis of Swing Buttons

Buttons are about the simplest user interface elements, so they are a good place to become comfortable with the *model-view-controller* pattern. For most components, the model class implements an interface

whose name ends in *Model*. Thus, there is an interface called **ButtonModel**. Classes implementing that interface can define the state of the various kinds of buttons. Actually, buttons aren't all that complicated, and the Swing library contains a single class, called **DefaultButtonModel**, that implements this interface.

<code>getActionCommand()</code>	The action command string associated with this button
<code>getMnemonic()</code>	The keyboard mnemonic for this button
<code>isArmed()</code>	true if the button was pressed and the mouse is still over the button
<code>isEnabled()</code>	true if the button is selectable
<code>isPressed()</code>	true if the button was pressed but the mouse button hasn't yet been released
<code>isRollover()</code>	true if the mouse is over the button
<code>isSelected()</code>	true if the button has been toggled on (used for check boxes and radio buttons)

Table: The accessor methods of the **ButtonModel** interface

Each **JButton** object stores a button model object, which we can retrieve.

```
JButton button = new JButton("Blue");
ButtonModel model = button.getModel();
```

9.10. Building GUI with Swing

The following examples demonstrate the use of **JButton** in which whenever we click one of the buttons, the appropriate action listener changes the background color of the panel.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ButtonTest
{
    public static void main(String[] args)
    {
        ButtonFrame frame = new ButtonFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.show();
    }
}

/**
A frame with a button panel */
class ButtonFrame extends JFrame
{
    public ButtonFrame()
    {
        setTitle("ButtonTest");
        setSize(WIDTH, HEIGHT);

        // add panel to frame

        ButtonPanel panel = new ButtonPanel();
        Container contentPane = getContentPane();
        contentPane.add(panel);
    }

    public static final int WIDTH = 300;
    public static final int HEIGHT = 200;
}
```

```
/** A panel with three buttons. */
class ButtonPanel extends JPanel
{
    public ButtonPanel()
    {
        // create buttons

        JButton yellowButton = new JButton("Yellow");
        JButton blueButton = new JButton("Blue");
        JButton redButton = new JButton("Red");

        // add buttons to panel

        add(yellowButton);
        add(blueButton);
        add(redButton);

        // create button actions

        ColorAction yellowAction = new ColorAction(Color.yellow);
        ColorAction blueAction = new ColorAction(Color.blue);
        ColorAction redAction = new ColorAction(Color.red);
        // associate actions with buttons

        yellowButton.addActionListener(yellowAction);
        blueButton.addActionListener(blueAction);
        redButton.addActionListener(redAction);
    }
}

/** An action listener that sets the panel's background color. */
private class ColorAction implements ActionListener
{
    public ColorAction(Color c)
    {
        backgroundColor = c;
    }
    public void actionPerformed(ActionEvent event)
    {
        setBackground(backgroundColor);
        repaint();
    }
    private Color backgroundColor;
}
}
```